Assignment 2 Write Up
Sherri Son (cs8607@nyu.edu)

Problem (transformer_accounting): Transformer LM resource accounting (5 points)
(a) Consider GPT-2 XL, which has the following configuration:
vocab_size : 50,257
context_length : 1,024
num_layers : 48
d_model : 1,600
num_heads : 25
d_ff : 6,400
Suppose we constructed our model using this configuration. How many trainable parameters would our model have? Assuming each parameter is represented using single-precision floating point, how much memory is required to just load this model?

Answer :
:https://colab.research.google.com/drive/1h4zj7VowCqr71Q2CYet9N5Xxz5-DJYrK?usp=sharing
Parameter is a number that the model learns during training. We need to sum all parameter contributions across all components, this is simply written as by this formula, which is P = 2*vocab_size*d_model + number of blocks L (4*d_model^2 + 3*d_model*_d_ff + 2*d_model) + d_model
Calculating with this formula, the answer would be 2,127,057,600 parameters. And then, each parameter is stored as one float32 number, and float32 uses 32bits = 4bytes. So, overall, 2,127,057,600 * 4 = 8,508,230,400 bytes is needed (8.508GB)

(b) Identify the matrix multiplies required to complete a forward pass of our GPT-2 XL-shaped model. How many FLOPs do these matrix multiplies require in total? Assume that our input sequence has context_length tokens.
Deliverable: A list of matrix multiplies (with descriptions), and the total number of FLOPs Required.

Answer:
:https://colab.research.google.com/drive/1h4zj7VowCqr71Q2CYet9N5Xxz5-DJYrK?usp=sharing
In my code, the matrix multiplication occur in <q_proj>, <k_proj>, <v_proj>, the two operations in <scaled_dot_product_attention>, <output_proj>, <SwiGLU's w1, w3, and w2>, and the final <lm_head>.
Total number of FLOP = number of blocks * (4 attention linear layers + 2 attention @ operations + 3 SwiQLU Linear Lyaers) + Final Vocabulary L
=> L * (8SD^2 + 4S^2D + 6SDF) + 2SDV
=> 4,513,336,524,800 FLOPs.

*L=num_layers(Number of transfer Blocks) / S = x.shape[-2] inside a block/(Number of tokens in the input sequence) / D = d_model(Features per token) /F = d_ff(Features inside SwiGLU) / V = vocab_size(Number of possible output tokens)

(c) Based on your analysis above, which parts of the model require the most FLOPs?
Deliverable: A one-to-two sentence response.
:https://colab.research.google.com/drive/1h4zj7VowCqr71Q2CYet9N5Xxz5-DJYrK?usp=sharing
From the calculations of 1(b), SwiGLU requires the most FLOPs with 66.91%, followed by Attention Linear Layers.

(d) Repeat your analysis with GPT-2 small (12 layers, 768 d_model, 12 heads), GPT-2 medium (24layers, 1024 d_model, 16 heads), and GPT-2 large (36 layers, 1280 d_model, 20 heads). As the model size increases, which parts of the Transformer LM take up proportionally more or less of the total FLOPs?
Deliverable: For each model, provide a breakdown of model components and its associated FLOPs (as a proportion of the total FLOPs required for a forward pass). In addition, provide a one-to-two sentence description of how varying the model size changes the proportional FLOPs of each component.

Answer:
:https://colab.research.google.com/drive/1h4zj7VowCqr71Q2CYet9N5Xxz5-DJYrK?usp=sharing
: As model size increases, attention projections and SwiGLU take up larger proportions of total FLOP's.

(e) Take GPT-2 XL and increase the context length to 16,384. How does the total FLOPs for one forward pass change? How do the relative contribution of FLOPs of the model components change?
Deliverable: A one-to-two sentence response.

Answer:
:https://colab.research.google.com/drive/1h4zj7VowCqr71Q2CYet9N5Xxz5-DJYrK?usp=sharing
Increasing context length from 1,024 to 16,384 raises total FLOPs from 4,513,336,524,800 to 149,522,795,724,800, approximately 33.13 times, because attention products scale quadratically with sequence length. However, the SwiGLU percentage has decreased from 66.91% to 32.32% and attention products are contributing the most here as 55.15%.

Problem (learning_rate_tuning): Tuning the learning rate (1 point)
As we will see, one of the hyperparameters that affects training the most is the learning rate. Let's see that in practice in our toy example. Run the SGD example above with three other values for the learning rate: 1e1, 1e2, and 1e3, for just 10 training iterations. What happens with the loss for each of these learning rates? Does it decay faster, slower, or does it diverge (i.e., increase over the course of training)?
Deliverable: A one-two sentence response with the behaviors you observed.

Answer:
: With learning rate 10, loss decreases faster than the learning rate 1 baseline, falling from 26.27 to 3.53; with learning rate 100, it initially stays unchanged and then decreases fastest, reaching nearly zero. With learning rate 1000, loss diverges during the 10-iteration run, increasing from 26.27 to approximately. The loss diverges, increasing sharply during training.
*1e1, 1e2, and 1e3 = 10/100/1000

```
Learning rate: 1
Step 0: loss = 26.2714
Step 1: loss = 25.2311
Step 2: loss = 24.5225
Step 3: loss = 23.9594
Step 4: loss = 23.4826
Step 5: loss = 23.0644
Step 6: loss = 22.6893
Step 7: loss = 22.3476
Step 8: loss = 22.0327
Step 9: loss = 21.7399
```

```
Learning rate: 100
Step 0: loss = 26.2714
Step 1: loss = 26.2714
Step 2: loss = 4.50746
Step 3: loss = 0.107874
Step 4: loss = 1.10268e-16
Step 5: loss = 1.229e-18
Step 6: loss = 4.13848e-20
Step 7: loss = 2.46532e-21
Step 8: loss = 2.11491e-22
Step 9: loss = 2.3499e-23
```

```
Learning rate: 10
Step 0: loss = 26.2714
Step 1: loss = 16.8137
Step 2: loss = 12.3943
Step 3: loss = 9.69725
Step 4: loss = 7.85477
Step 5: loss = 6.51251
Step 6: loss = 5.49243
Step 7: loss = 4.69344
Step 8: loss = 4.05316
Step 9: loss = 3.53075
```

```
Learning rate: 1000
Step 0: loss = 26.2714
Step 1: loss = 9483.98
Step 2: loss = 1.63803e+06
Step 3: loss = 1.82214e+08
Step 4: loss = 1.47593e+10
Step 5: loss = 9.31481e+11
Step 6: loss = 4.78192e+13
Step 7: loss = 2.05739e+15
Step 8: loss = 7.58308e+16
Step 9: loss = 2.43501e+18
```



Problem (adamwAccounting): Resource accounting for training with AdamW (2 points)
Let us compute how much memory and compute running AdamW requires. Assume we are using float32 for every tensor.

(a) How much peak memory does running AdamW require? An algebraic expression for each of parameters, activations, gradients, and optimizer state, as well as the total.

Answer:
*B = batch_size, V = vocab_size, S = context_length, L = num_ layers, D= d_model, and H = num_heads
* Each float 32 occupies by 4 bytes

:
Parameter = 2VD + L(16D^2+2D) + D
Parameter memory = 4P bytes
Activation memory = 4B[L(16SD + 2HS^2) + SD + 2SV] bytes
Gradient memory = 4P bytes
Optimizer-state memory = 8P bytes

: Total memory = 16P + 4B [L(16SD + 2HS^2) +SD+2SV] bytes

Verification by code:

```
Two RMSNorm outputs: 2 * B*S*D
Q, K, V projections: 3 * B*S*D
Weighted sum of values: 1 * B*S*D
Attention output: 1 * B*S*D
Feed-forward W1 output: 4 * B*S*D
SiLU output: 4 * B*S*D
Feed-forward W2 output: 1 * B*S*D
Attention scores: B*H*S^2
Attention probabilities: B*H*S^2

Total per block: 16*B*S*D + 2*B*H*S^2

P = number of trainable parameters
P = 2*V*D + L*(16*D^2 + 2*D) + D

A = number of stored activation values
A = B * [L*(16*S*D + 2*H*S^2) + S*D + 2*S*V]

Memory — Float32 uses 4 bytes per values
Parameters:      4*P bytes
Activations:     4*A bytes
Gradients:       4*P bytes
Optimizer state: 8*P bytes  (AdamW stores m and v)
Total:          16*P + 4*A bytes

Expanded total:
16*(2*V*D + L*(16*D^2 + 2*D) + D) + 4*(B * [L*(16*S*D + 2*H*S^2) + S*D + 2*S*V]) bytes
```

(b) Instantiate your answer for a GPT-2 XL-shaped model to get an expression that only depends on the batch_size. What is the maximum batch size you can use and still fit within 80GB memory?
Deliverable: An expression that looks like a·batch_size + b for numerical values a, b, and a number representing the maximum batch size.

Answer: Integer 2

: GPT- 2 XL shaped model has 48 blocks, width 1600, 25 heads, context length 1024, and 50,257 of vocabulary size. (V = 50,257 , S = 1024, L = 48, D = 1600, H=25) In order to answer this question, we need to put those numbers into the formula that I mentioned earlier. The model has P = 2,127,057,600 parameters, so its parameters, gradients, and AdamW state require 16P = 34,032,921,600 bytes of fixed memory. The activation memory per batch item is: a = 4[L(16SD + 2HS^2) + SD + 2SV] = 15,517,753,344 Bytes.

: Therefore, M(B) = 15.517 * Batch Size + 34.032 GB

: (80 - 34.032GB) / 15.517 = 2.962
B <= 2.962

: The maximum integer batch size is 2. If the batch size exceeds 2, it will be over budget 80GB.

(c) How many FLOPs does running one step of AdamW take?
Deliverable: An algebraic expression, with a brief justification.

Answer: AdamW FLOPs = 14P = 14[2VD + L(16D^2+ 2D) + D]

: AdamW equations require approximately 14 operations per parameter : 3 for updating the first moment, 4 for updating second moment, 5 for the adaptive parameter update, and 2 for weight decay. Ignoring small per-tensor calculations.
: AdamW FLOPs = 14P = 14[2VD + L(16D^2+ 2D) + D]

(d) Model FLOPs utilization (MFU) is defined as the ratio of observed throughput (tokens per second) relative to the hardware's theoretical peak FLOP throughput [Chowdhery et al., 2022]. An NVIDIA A100 GPU has a theoretical peak of 19.5 teraFLOP/s for float32 operations. Assuming you are able to get 50% MFU, how long would it take to train a GPT-2 XL for 400K steps and a batch size of 1024 on a single A100? Following Kaplan et al. [2020] and Hoffmann et al. [2022], assume that the backward pass has twice the FLOPs of the forward pass.
Deliverable: The number of days training would take, with a brief justification.

:Around 6,583 days

:From <Problem (transformer_accounting) (b)>, we calculated processing one 1024 token sequence costs 4,513,336,524,800 FLOPs.

:Backward pass = twice the FLOPs of the forward pass
: Training flops per sequence = Forward pass + backward pass = 3 * the cost of forward pass
: total flops = training flops per sequence * batch size * steps
: flops per second = 0.50 * 19.5 * 10^12 = 9750000000000 Flop/s
: Seconds = Total flops / flops per second
: Days = seconds / (60*60*24)

*Detailed Calculation in the code

Problem (first_training_run): Put it together (3 points)
Deliverable: Write a short script (a notebook is fine too) that:

4. Periodically prints (or plots) the training loss.

Deliverable: A learning curve (loss on the y-axis, batches processed on the x-axis) showing loss going down. Write 1-2 sentences on what you observed: does it decrease smoothly? Feel free to also use decode (implemented next) to look at what your model outputs before vs. after training, though we aren't expecting fluent text out of a model this small / undertrained.)



Answer: Training loss decreased rapidly at first, then declined more gradually and began to level off. Although the overall trend was downward, small batch to batch fluctuations meant the decrease was not perfectly smooth.

Problem (decoding): Decoding (3 points)
Deliverable: Implement a function to decode from your language model. We recommend that you support the following features:
• Generate completions for a user-provided prompt (i.e., take in some x1...t and sample a completion until you hit an <|endoftext|> token).
• Allow the user to control the maximum number of generated tokens.
• Given a desired temperature value, apply softmax temperature scaling to the predicted next-word distributions before sampling.
• Top-p sampling (Holtzman et al., 2020; also referred to as nucleus sampling), given a user-specified threshold value. Indicate in your writeup where this script lives.

Answer: The decoding function is implemented in student/generate.py